Principal Components & Intro to Non-Linear Solvers

Overview

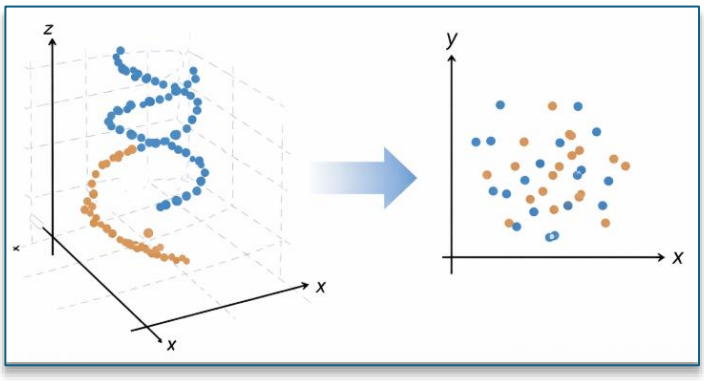
- **approximate tensors using the principal components**
- **create a k dimensional vectors using the first principal components to describe data**
- **approximate a function using an n-th order Taylor series with remainder**
- **Newtons method for scalar equations**
- **the secant method**
- **the convergence properties.**

Principal Components

Principal Components

Motivating Example

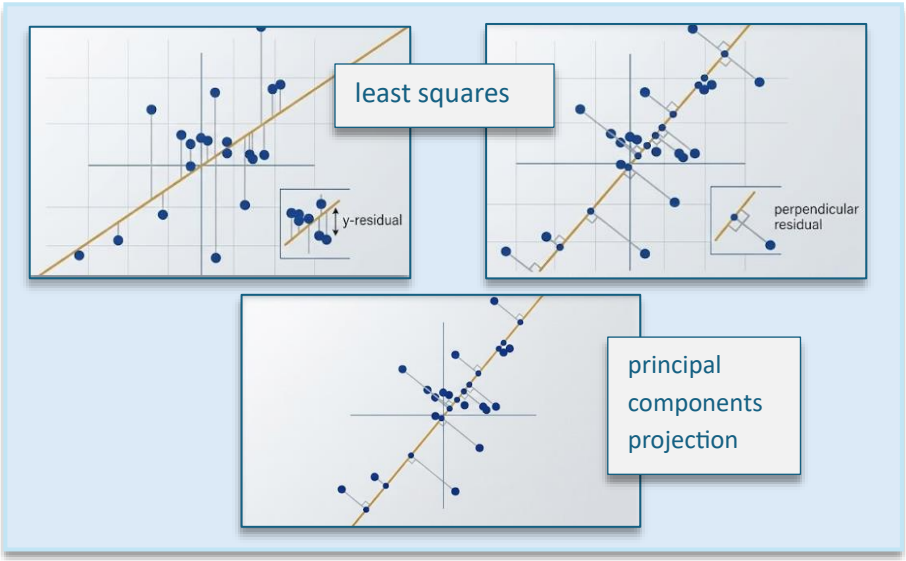
- often when collapsing a **higher dimensional** vector space **into** a **lower dimensional** representation, **structure** inherent to the data **is lost** or unrecognizable



ChatGPT 5.3 Instant 1

PCA vs Least Squares

- **principal** components
  - **unsupervised** learning
  - finds **directions** (components) of **maximum variance** in predictors
- **vertical** least squares
  - **supervised** learning
  - **minimizes** squared **vertical distances** to predict $\boldsymbol{Y}$ from $\boldsymbol{X}$
- **orthogonal** least squares
  - **supervised** learning
  - **minimizes** the sum of squared **perpendicular distances**



Gemini 3

Numerics

- take the **SVD** of the **data** set

$$\boldsymbol{D} = \boldsymbol{\sigma_1 u_1 v_1^T} + \boldsymbol{\sigma_2 u_2 v_2^T}$$

"**best**" in the sense of **maximizing** the *first* term

the **best** one rank **approximation** is the **first term**

$$\| \boldsymbol{D} \|_F^2 = \| \boldsymbol{\sigma_1 u_1 v_1^T} \|_F^2 + \| \boldsymbol{\sigma_2 u_2 v_2^T} \|_F^2$$

which means **minimizing** the *next* term

```
julia> D
20×2 Matrix{Float64}:
 0.0632719  -1.659
 0.505667    0.376497
 1.55698     1.27546
 ⋮
 1.12682     0.0639133
-0.156161    0.402374
```

```
julia> fact = svd(D);

julia> U = fact.U; Vt = fact.V; S = fact.S;

julia> S[1]*U[:,1]*transpose(Vt[1,:])
20×2 Matrix{Float64}:
 -0.797864   0.797864
  0.441082  -0.441082
  1.41622   -1.41622
 ⋮
  0.595365  -0.595365
  0.123107  -0.123107

julia> S
2-element Vector{Float64}:
 5.609571973558723
 2.8092564856735898
```

rank one approximation:
$v_1 \longrightarrow$ principle **direction** in feature space
$u_1\sigma_1 \rightarrow$ **coordinates** of observations along the direction v_1

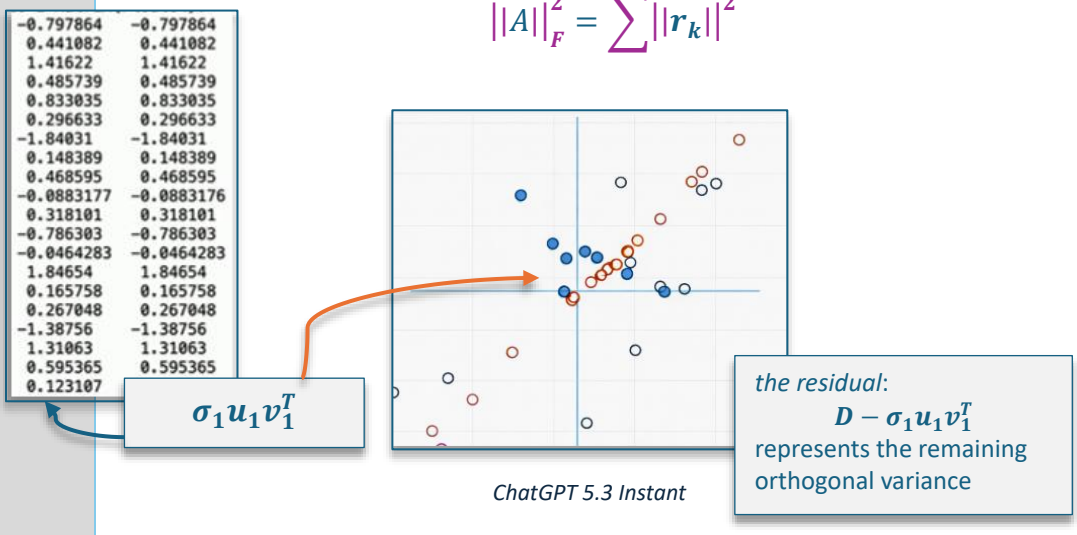
The First Term

- the **first** term in

$$\| D \|_F^2 = \| \sigma_1 u_1 v_1^T \|_F^2 + \| \sigma_2 u_2 v_2^T \|_F^2$$

- **Frobenius norm squared** is the sum of the two norms of each row squared

$$||A||_F^2 = \sum ||r_k||^2$$



ChatGPT 5.3 Instant

the residual:
$D - \sigma_1 u_1 v_1^T$
represents the remaining orthogonal variance

The Second Term

- the **second term**

$$\sigma_2\, u_2\, v_2^T$$

- the **remaining SVD terms** represent the **residual** structure **not captured** by the **first** principal component

The 3-D Case

- find the best **rank 1** and **2**

separate a matrix D into **rank one** matrices

$$D = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \sigma_3 u_3 v_3^T$$

$$\| D \|_F^2 = \| \sigma_1 u_1 v_1^T \|_F^2 + \| \sigma_2 u_2 v_2^T + \sigma_3 u_3 v_3^T \|_F^2$$

best **rank 1**matrix

$$\| D \|_F^2 = \| \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T \|_F^2 + \| \sigma_3 u_3 v_3^T \|_F^2$$

best **rank 2** matrix

$$D = U \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{bmatrix} V^T$$

```
julia> D
1000×3 Matrix{Float64}:
5.43274   7.90601   1.91714
1.07729   5.89151   3.75612
⋮
1.2538    4.87763  10.9128

julia> fact = svd(D)

julia> U = fact.U

julia> S = fact.S

julia> Vt = fact.Vt

julia> S1 = Diagonal([S[1], 0, 0])

julia> S2 = Diagonal([S[1], S[2], 0])

julia> rank1 = U*S1*Vt

julia> rank2 = U*S2*Vt

julia> using DelimitedFiles

julia> writedlm("/tmp/rank1.csv", rank1, ',')

julia> writedlm("/tmp/rank2.csv", rank2, ',')
```

Intro to Nonlinear Solvers

Nonlinear Solvers in 1-D

Taylor Series

- from calculus, we know that for smooth functions

Taylor Series

$$f(x) = f(a) + f'(a)(x-a) + \left(\frac{f''(a)}{2!}\right)(x-a)^2 + \left(\frac{f'''(a)}{3!}\right)(x-a)^3 + \cdots$$

- this typically only holds inside a convergence radius R

$|x - a| < R$

- if the derivatives exist and are continuous then,

$$f(x) = f(a) + f'(a)(x-a) + \cdots + \left(\frac{f^{(n)}(a)}{n!}\right)(x-a)^n + \left(\frac{f^{(n+1)}(\xi)}{(n+1)!}\right)(x-a)^{(n+1)}$$

Taylor Series "with a remainder"

truncate after the n-th power

Common Examples of Taylor Series

Exponential Taylor Series

$$exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \cdots$$

Sine Taylor Series

$$sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

Cosine Taylor Series

$$cos(x) = 1 - \frac{x^2}{2} + \frac{x^4}{4!} - \cdots$$

Complex Exponential Expansion

$$exp(ix) = 1 + ix + \frac{(ix)^2}{2} + \frac{(ix)^3}{3!} + \frac{(ix)^4}{4!} + \frac{(ix)^5}{5!} + \cdots$$

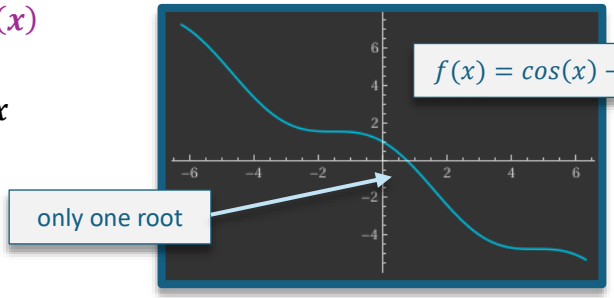
Euler's Identity

$$exp(ix) = 1 + ix - \frac{x^2}{2} - i\frac{x^3}{3!} + \frac{x^4}{4!} + i\frac{x^5}{5!} - \cdots = cos(x) + i\,sin(x)$$

Finding Roots

- How to solve $x = \cos(x)$

$$f(x) = cos(x) - x$$



- **approximate** the root

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

- find the **exact root** of the **linearized model**

$$f(x_0) + f'(x_0)(x - x_0) = 0$$

$$x - x_0 = -f(x_0)/f'(x_0)$$

Newton's Method

- **use** as **next** 'guess'

$$x_1 = x_0 - f(x_0)/f'(x_0)$$

Implementation

- **Newton's Method** / Newton–Raphson iterates

with initial guess 0, the iteration converges quickly

$$x_{(n+1)} = x_n - f(x_n)/f'(x_n)$$

$$f(x) = cos(x) - xf'(x) = -sin(x) - 1$$

second order convergence

- **double** the number of **digits** in **each step**

$$e_{(n+1)} \approx Ce_n^2$$

error in the next step (n+1), is previous error squared times a constant C

```
julia> f(x) = cos(x) - x
f (generic function with 1 method)

julia> fp(x) = -sin(x) - 1
fp (generic function with 1 method)

julia> x0 = 0
0

julia> x1 = x0 - f(x0)/fp(x0)
1.0

julia> x2 = x1 - f(x1)/fp(x1)
0.7503638678402439
```

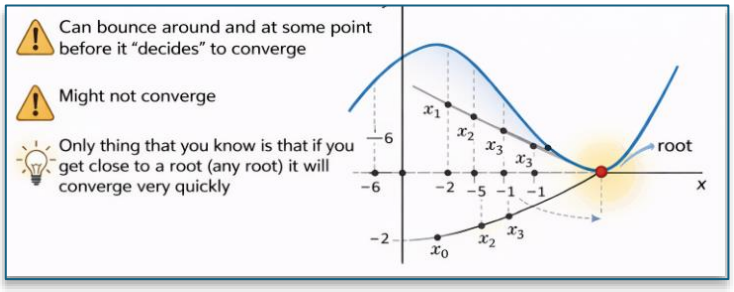
```
julia> x3 = x2 - f(x2)/fp(x2)
0.7391128909113617

julia> x4 = x3 - f(x3)/fp(x3)
0.739085133385284

julia> x5 = x4 - f(x4)/fp(x4)
0.7390851332151607
```

```
julia> x2 - x5
0.01127873462508322

julia> x3 - x5
2.775769620100288e-5

julia> x4 - x5
1.7012335984389892e-10
```

When Convergence is Messy

- sometimes **convergence** can "bounce around" – **taking** some **time** to settle into a **convergent** pattern

$$x_{(n+1)} = x_n - \frac{f(x_n)}{f'(x_n)}$$



ChatGPT 5.3 Thinking

```
julia> x = x0
-6

julia> x = x - f(x)/fp(x)
-0.5598827773710564

julia> x = x - f(x)/fp(x)
2.441099878014685

julia> x = x - f(x)/fp(x)
0.49191148608506574
```

```
julia> x = x - f(x)/fp(x)
0.7564751611987038

julia> x = x - f(x)/fp(x)
0.7391506975713239

julia> x = x - f(x)/fp(x)
0.7390851341642681

julia> x = x - f(x)/fp(x)
0.7390851332151607
```

Secant Method

- there are some **problems** with **Newton's** method
  - you need **both** $f(x)$and $f'(x)$
  - can have a **complicated convergence** pattern
- solution is to **approximate** the **derivative** by **taking** the **slope** between two points

$$x_{(n+1)} = \frac{x_n - f(x_n)}{\left(f(x_n) - f\left(x_{(n-1)}\right)/\left(x_n - x_{(n-1)}\right)\right)}$$

- convergence is **slower**, but only a **single function** is needed at **each step**
- similar to **Newton's** method, if it **starts** to close **convergence accelerates**

```
julia> f(x) = cos(x) - x
f (generic function with 1 method)

julia> x0 = 0; x1 = 1; x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0))
0.6850733573260451

julia> x0 = x1; x1 = x2; x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0))
0.736298997613654

julia> x0 = x1; x1 = x2; x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0))
0.739119361912693

julia> x0 = x1; x1 = x2; x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0))
0.739085112274639

julia> x0 = x1; x1 = x2; x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0))
0.7390851332151607
```